

METAGPT: META PROGRAMMING FOR A MULTI-AGENT COLLABORATIVE FRAMEWORK

Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, Jurgen Schmidhuber

DeepWisdom, AI Initiative, King Abdullah University of Science and Technology, Xiamen University, The Chinese University of Hong Kong, Shenzhen, Nanjing University, University of Pennsylvania, University of California, Berkeley, The Swiss AI Lab IDSIA/USI/SUPSI

May 24, 2026

Navigation icons

Hong et al. (DeepWisdom)

MetaGPT: Meta Programming...

May 24, 2026

1 / 21

Background: LLM-based agents are transforming software development

- We leverage LLMs that can plan, code, use tools, and reason in natural language.
- We build on multi-agent systems to seek scalable, end-to-end automation.
- We observe strong industry adoption of coding copilots and research gains from agent collaboration.

Navigation icons

Hong et al. (DeepWisdom)

MetaGPT: Meta Programming...

May 24, 2026

2 / 21

Background: Why structure matters in real engineering

- We mirror how human teams rely on SOPs and role specialization.
- We standardize artifacts (PRDs, designs, tests) to reduce ambiguity.
- We translate structured workflows into agents to improve reliability.

Navigation icons

Hong et al. (DeepWisdom)

MetaGPT: Meta Programming...

May 24, 2026

3 / 21

The Problem: Unstructured agent chats don't scale

- We observe cascading hallucinations that cause logic inconsistencies.
- Naive LLM chaining lacks verification and alignment.
- Point-to-point dialogues are brittle and overload context.

Navigation icons

Hong et al. (DeepWisdom)

MetaGPT: Meta Programming...

May 24, 2026

4 / 21

- We face ambiguous requirements and shifting scope.
- We see poor handoffs across roles and stages.
- We need runtime checks for functional correctness.

- Shared message pool: we log PRD, design, code, and tests as structured messages (global memory).
- Publish-subscribe: PM, Architect, Engineer, and QA subscribe to role-relevant artifacts for targeted retrieval.
- SOP artifacts: we standardize PRD → system design → task plan → code → tests to keep handoffs aligned.
- Executable feedback: we run code/tests with the Engineer agent and update artifacts based on failures, closing the loop.



Figure: B.2 MetaGPT 开发流程

- Roles: PM, Architect, Project Manager, Engineer, QA.
- We produce standardized artifacts and handoffs at each stage.
- We reduce ambiguity and drift via an assembly line workflow.

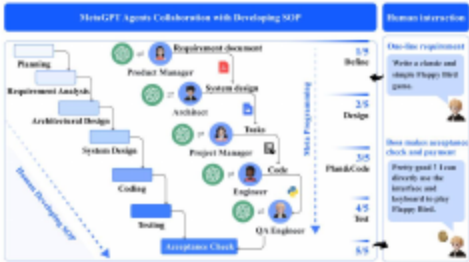


Figure: MetaGPT与人类团队开发SOP对比：任务分解与角色协作

- PM: PRD with goals, stories, and requirements.
- Architect: system design, interfaces, and diagrams.
- Project Manager: task breakdown and file assignments.
- Engineer + QA: implement, test, and verify against artifacts.



Figure: MetaGPT软件开发流程示意，突出对SOP的依赖；详见附录B

- We publish a PRD; downstream roles subscribe to requirements and constraints.
- We translate the PRD into file structure, data models, and interfaces.
- We decompose work into tasks with clear owners and deliverables.

Figure 8: Data structures and interface definitions for the “Drawing App” generated by the *architect* agent.

Figure: 项目经理将项目分解为任务，并按功能解析代码文件为独立任务指派工程师

- We implement per interfaces and run tests for executable feedback.
- We verify interfaces and workflows; failures are fed back to Engineer for bounded iterative fixes.
- We maintain consistent end-to-end traceability via standardized artifacts.



Figure: B.2 MetaGPT开发流程：从实现与测试到迭代修正

Our experimental setup: datasets, metrics, baselines

- Code generation: HumanEval, MBPP; Projects: SoftwareDev.
- Metrics: Pass@1, executability, human revision cost, productivity.
- Baselines: GPT-4 and ChatDev.
- Full baseline citations appear on the References slide.

Our results in one slide: we show stronger single-try accuracy and executability

- We achieve Pass@1 of 85.9% on HumanEval and 87.7% on MBPP, surpassing strong single-agent and multi-agent baselines.
- Our executable feedback contributes +4.2 pts (HumanEval) and +5.4 pts (MBPP).
- On SoftwareDev, we reach executability 3.75/4 with 0.83 human revision cost, outperforming ChatDev at comparable time.
- Takeaway: Structure (SOPs + roles + message pool) + runtime checks improves reliability over chat-based agents.

- Our quality depends on backend LLM strength.
- Complex projects may require more retries.
- SOPs may require domain-specific tuning.

- We use SOPs + roles + structured messaging to reduce hallucinations.
- We use executable feedback to raise functional correctness.
- We improve executability, quality, and efficiency.



Figure: B.2 MetaGPT开发流程

- We will explore self-improving role constraints and dynamic SOPs.
- We plan broader toolsets: UI, multimodal, new runtimes.
- We envision agent marketplaces for regulated, reusable workflows.

- OpenAI. GPT-4 Technical Report (2023).
- Qian et al. Communicative Agents for Software Development (ChatDev), 2023.

- Thank you for your attention!
- Questions and feedback are welcome.

- Thanks to collaborators, open-source contributors, and the research community.
- We acknowledge benchmarks and datasets used in this work.

Backup: Qualitative demos—end-to-end, multi-file, testable apps

- We autonomously build games, GUIs, and data apps.
- We document standardized artifacts at each step.
- We demonstrate practical, reproducible outputs.



Figure: Demo softwares developed by MetaGPT.

Backup: Case study—Drawing App pipeline outcome

- We use artifacts to guide implementation and testing.
- The resulting UI runs as expected.
- This validates pipeline coherence end-to-end.



Figure: The “Drawing App” generated by MetaGPT.

Backup: Ecosystem—AgentStore enables reusable SOP agents

- We compose and configure compliant agents.
- We share role specializations and workflows.
- We catalyze community-driven agent ecosystems.

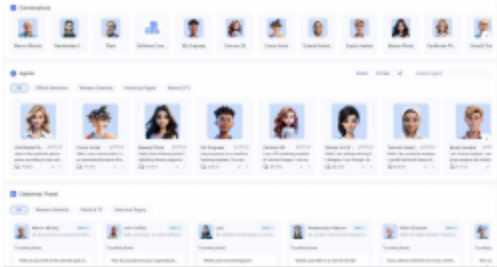


Figure: AgentStore: MetaGPT框架内的代理创建与管理平台